Preliminar de arquitectura: Procesamiento distribuido de base de datos SECOP II y RAG híbrido

Propuesta técnica del proyecto

1. Propósito y alcance

El proyecto propone una plataforma para analizar grandes volúmenes de información de contratación pública de SECOP II mediante un motor de procesamiento distribuido propio y utilizar los resultados en un sistema de consulta en lenguaje natural.

Para que cualquier persona tenga acceso a la información de contratación pública en lenguaje natural, se busca calcular indicadores que permitan identificar **señales de riesgo** en los procesos de contratación y facilitar consultas estadísticas y explicativas.

Se trabajará principalmente con dos conjuntos de datos: *Procesos de Contratación*, que contiene información de participantes, invitados e interesados, y *Contratos Electrónicos*, que registra contratos adjudicados, proveedores ganadores, entidades, valores, categorías UNSPSC y modalidades.

La solución integra dos capacidades: (1) procesamiento paralelo mediante Go y MapReduce; y (2) un RAG híbrido que combina consultas SQL con búsqueda semántica. Los indicadores serán herramientas de priorización y revisión: por sí solos no constituyen prueba de corrupción o ilegalidad.

2. Arquitectura y flujo general

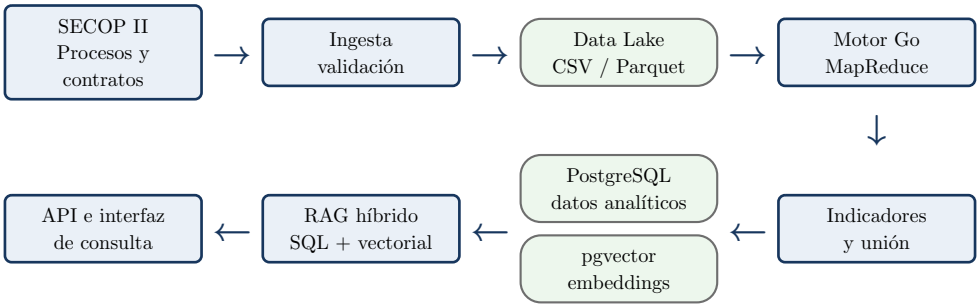


Figura 1: Vista general de los componentes y del recorrido de los datos.

El flujo comienza con la descarga y validación de SECOP II. Los datos originales y normalizados se conservan en un Data Lake; después se dividen en particiones para procesarlas en paralelo. El motor calcula indicadores y genera resultados estructurados. Estos resultados se cargan en PostgreSQL y, cuando contienen explicaciones o evidencia textual útil, también se convierten en documentos vectoriales en pgvector. Finalmente, el RAG selecciona la fuente adecuada para responder cada pregunta.

3. Ingesta, almacenamiento y preparación

Un servicio de ingesta obtiene los conjuntos de datos, verifica columnas y tipos, normaliza fechas e identificadores, identifica duplicados y registra la fecha y versión de cada extracción. Se conservarán los archivos originales para trazabilidad y para poder repetir los análisis.

Se propone un Data Lake, por ejemplo MinIO compatible con S3, con tres zonas:

- **Raw:** archivos originales sin modificar.
- **Processed:** datos depurados y normalizados, preferiblemente en Parquet.
- **Results:** resultados intermedios y finales de las ejecuciones distribuidas.

Parquet facilita el procesamiento analítico por columnas. Un componente *Chunker* divide los archivos en particiones manejables, que serán las unidades de trabajo asignadas a los Workers.

4. Motor de procesamiento distribuido

El motor se implementará en Go con arquitectura **Master–Worker**, comunicación gRPC y mensajes definidos mediante Protocol Buffers. El Master coordina las tareas; los Workers procesan las particiones.

- **Master:** divide y asigna tareas, mantiene su estado, coordina el Shuffle, recibe resultados y detecta fallos mediante *heartbeats*.
- **Workers:** leen la partición asignada, ejecutan Map o Reduce, escriben resultados y reportan su estado.
- **Map:** transforma registros en pares clave–valor.
- **Shuffle:** agrupa y redistribuye los pares por clave para reunir registros relacionados.
- **Reduce:** procesa cada grupo y calcula agregaciones o indicadores.

Si un Worker deja de responder, el Master podrá reasignar sus tareas pendientes. Las salidas deben identificarse por ejecución y tarea para evitar contabilizar dos veces un resultado. Docker empaqueta los servicios y Kubernetes administra su despliegue y reinicio; Kubernetes no reemplaza al Master, pues este último controla las tareas MapReduce.

Estado actual del repositorio: ya están implementados el Chunker (indexación por offsets del CSV), la cola de tareas en el Master, los heartbeats con reasignación de tareas ante timeout, el Shuffle entre Workers, los Jobs A, B1 y B2 y el join de la Etapa 2, validados contra un oráculo secuencial en Python. También existen los manifiestos de Kubernetes, la publicación de imágenes en GHCR y el punto de entrada del front (Sección 8). Quedan pendientes:

- ejecutar el join como tarea distribuida, pues hoy se calcula dentro del Master;
- persistir los resultados, que hoy solo se reportan en el log del Master;
- volver a tres Workers en el clúster, ya que el StatefulSet corre temporalmente con dos réplicas;
- todos los componentes de este documento posteriores al motor: PostgreSQL, pgvector, RAG, API e interfaz.

5. Cálculo de indicadores y unión de resultados

El diseño contempla tres trabajos independientes:

1. **Job A – Competencia:** calcula la relación entre proveedores que respondieron y proveedores invitados:
$$I_c = \frac{\text{proveedores que respondieron}}{\text{proveedores invitados}}.$$
Debe controlar denominadores cero, valores faltantes y duplicados.
2. **Job B1 – Desviación de precios:** compara el valor de un contrato con la mediana de referencia de su categoría UNSPSC:
$$D_p = \frac{V_{\text{contrato}} - \text{Mediana}\big(V_{\text{categoría}}\big)}{\text{Mediana}\big(V_{\text{categoría}}\big)} \times 100.$$
La comparación debe considerar contratos suficientemente similares. No se deben promediar medianas locales de Workers para obtener la mediana global; se requiere un cálculo distribuido correcto o un método de cuantiles aproximados documentado.
3. **Job B2 – Concentración:** mide qué porcentaje de los contratos de un proveedor proviene de una misma entidad, como lo define la propuesta (`00-propuesta`) y lo implementa el código:
$$C_p = \frac{\text{contratos del proveedor con la entidad}}{\text{total de contratos del proveedor}} \times 100.$$
La variante centrada en la entidad (qué porcentaje de los contratos de una entidad gana un mismo proveedor) mide otra cosa: dependencia de la entidad frente a un proveedor. Queda como alternativa por evaluar (Sección 9).

Los resultados se relacionan mediante un **Reduce Side Join** usando el identificador del proceso. El Shuffle agrupa los registros de los distintos Jobs por esa clave y Reduce combina sus indicadores. Se propone una unión externa izquierda desde los procesos, para conservar también los que no tienen contrato adjudicado.

Hoy, los procesos sin contrato reciben valor cero en los indicadores ausentes y terminan con puntuación de riesgo alta. Antes de exponer cualquier ranking por el RAG se debe distinguir *dato ausente* de *valor cero*; de lo contrario, el sistema respondería que los procesos no adjudicados son los más riesgosos.

Opcionalmente, los indicadores pueden combinarse en un puntaje de priorización:
$$R = w_1 S_c + w_2 S_p + w_3 S_k,$$
donde los indicadores se normalizan y los pesos w_i se definen y justifican durante el proyecto. Los pesos y umbrales todavía no están establecidos. Se conservarán los valores individuales y sus fuentes, no solo el puntaje.

6. Persistencia e índice vectorial

El motor escribirá sus resultados en Parquet. Un proceso *Data Loader* validará los archivos y los cargará en una base analítica. Se propone comenzar con **PostgreSQL**, con tablas para procesos, contratos, proveedores, entidades, métricas de competencia, precios, concentración y alertas. Se conservarán todos los procesos y contratos, no únicamente los que generen alertas, para evitar sesgos en las consultas estadísticas.

La búsqueda vectorial complementa, pero no reemplaza, a SQL:

- **PostgreSQL:** conteos, sumas, filtros, agrupaciones, rankings y consultas por fecha.
- **pgvector:** búsqueda semántica de explicaciones, observaciones, metodología y fragmentos de documentos de soporte.

No es necesario generar embeddings para cada fila. Se construirán documentos textuales a partir de alertas y evidencias relevantes, se dividirán en fragmentos (*chunks*) y se transformarán en vectores mediante un modelo de

embeddings, por ejemplo Sentence Transformers. Cada fragmento conservará metadatos y referencias al proceso o contrato de origen.

7. RAG híbrido y acceso del usuario

El RAG híbrido decide cómo recuperar la información según la pregunta:

1. **Query Router:** clasifica la pregunta como estructurada, semántica o mixta.
2. **SQL Retriever:** consulta PostgreSQL para obtener cifras y agregaciones.
3. **Vector Retriever:** busca en pgvector explicaciones y documentos relacionados.
4. **Context Aggregator:** combina resultados y conserva sus referencias.
5. **LLM Generator:** redacta una respuesta basada en el contexto recuperado.

Por ejemplo, «¿qué empresa obtuvo más contratos en 2021?» requiere SQL. En cambio, «¿por qué esta empresa presenta señales de riesgo?» requiere consultar sus indicadores en SQL y recuperar explicaciones o evidencias relacionadas mediante búsqueda vectorial.

El RAG se expone mediante una API interna, por ejemplo FastAPI, que solo es accesible a través del front (Sección 8). Ni el navegador ni el front acceden directamente a las bases de datos. Las consultas SQL serán de solo lectura, con tablas permitidas, validación, límites y tiempos máximos. Las respuestas deberán mostrar cifras, periodo e identificadores de procesos o contratos cuando estén disponibles. Si no hay evidencia suficiente, el sistema deberá indicarlo y no inventar conclusiones.

8. Despliegue y acceso de usuarios

El objetivo es que cualquier persona conectada a la red de la universidad pueda usar el front, y que ningún otro componente sea alcanzable desde fuera del clúster.

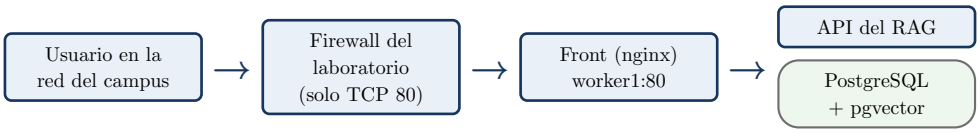


Figura 2: Único punto de entrada. Todo lo que está a la derecha del front es interno al clúster.

- **Un único punto de entrada:** el front corre en `worker1` con red de host y escucha en el puerto 80 de la IP física del nodo (manifiesto `k8s/front.yaml`). Se ubica en el mismo nodo que el Master y no en un Worker, para que las pruebas de tolerancia a fallos no lo afecten.
- **API detrás del front:** el servidor del front reenvía las rutas `/api/` a la API por la red interna del clúster. El navegador nunca se comunica con la API ni con las bases de datos.
- **Todo lo demás es interno:** API, PostgreSQL/pgvector, LLM, Master y Workers se publican solo como Services `ClusterIP`, sin `NodePort`, `LoadBalancer` ni `hostPort`. Las direcciones internas del clúster no son enrutables desde el campus.
- **Firewall:** desde la red del campus existe ruta hacia el segmento del laboratorio, pero el tráfico se descarta. Se solicitará al laboratorio de redes una única regla: permitir TCP 80 desde el campus hacia la IP de `worker1`. Por ahora se asume que esta apertura es posible.
- **Acceso:** los usuarios entran por `http://<IP de worker1>/`. Un nombre de dominio institucional requeriría un registro DNS gestionado por TI.

No se usa un *Ingress controller* porque solo se publica una aplicación en una única IP. Si más adelante se requiere HTTPS gestionado o publicar varias aplicaciones, se migrará a Ingress o Gateway API. El sistema no incluye autenticación (ver no-objetivos en `01-alcance`), por lo que el front y la API solo deben exponer información pública de SECOP II.

9. Decisiones abiertas

Las siguientes decisiones sobre el RAG se dejan explícitamente abiertas y se resolverán antes de implementar la persistencia:

1. **Catálogo de preguntas objetivo:** entre 15 y 20 preguntas reales, clasificadas como SQL, semánticas o mixtas. De este catálogo se derivan el esquema de PostgreSQL y lo que se vectoriza.
2. **Dimensiones descriptivas:** las salidas actuales del motor no incluyen fechas, ubicación, sector ni textos. Sin esos datos no se pueden responder preguntas como «¿qué empresa obtuvo más contratos en 2021?» o «procesos de riesgo alto en Chocó». Hay que decidir si esos datos se arrastran a través de los jobs o se cargan directamente desde los datos procesados.
3. **Qué se vectoriza:** las explicaciones de alertas se derivan de las mismas cifras que ya están en SQL y pueden generarse con plantillas. Los campos con texto libre real (`Objeto del Contrato`, `Descripción del Procedimiento`, `Justificación Modalidad de Contratación`) son los candidatos naturales para la búsqueda semántica.

4. **Datos personales:** Contratos Electrónicos incluye documentos de representantes legales, supervisores y ordenadores del gasto, además de datos bancarios. Se recomienda excluirlos de PostgreSQL, de los embeddings y del contexto del LLM.
5. **Embeddings como trabajo distribuido:** calcular los vectores de millones de descripciones es un cómputo paralelizable que podría ejecutarse como un Job del propio motor MapReduce.
6. **Definición de concentración:** centrada en el proveedor, como hoy, o en la entidad, o ambas.
7. **Componentes opcionales:** evaluar si MinIO y Airflow aportan lo suficiente frente a su costo de operación, o si bastan el sistema de archivos del nodo y un script del pipeline.

10. Operación, tecnologías y etapas

Airflow podrá coordinar las etapas periódicas de ingesta, preparación, ejecución MapReduce, carga SQL e indexación vectorial. No sustituye al Master: Airflow coordina el pipeline completo y el Master coordina las tareas internas del motor. Prometheus y Grafana permitirán monitorear duración, registros procesados, throughput, uso de CPU y memoria, fallos, reintentos y latencia del RAG.

Componente	Tecnología propuesta / función
Ingesta y preparación	Python; descarga, validación y normalización.
Almacenamiento	MinIO/S3 y Parquet; archivos originales, procesados y resultados.
Procesamiento	Go, MapReduce, gRPC y Protocol Buffers.
Despliegue	Docker y Kubernetes.
Persistencia	PostgreSQL para información estructurada.
Búsqueda semántica	pgvector y un modelo de embeddings.
RAG y API	Router, recuperadores SQL/vectoriales, LLM y FastAPI (interna).
Front y acceso	nginx en `worker1:80` como único punto de entrada desde el campus.
Operación	Airflow para orquestación; Prometheus y Grafana para monitoreo.

Tabla 1: Tecnologías propuestas por componente.

Plan de implementación: (1) ingesta y almacenamiento; (2) motor MapReduce genérico y tolerancia a fallos; (3) Jobs A, B1, B2 y Reduce Side Join; (4) persistencia en PostgreSQL; (5) documentos y embeddings en pgvector; (6) RAG híbrido; (7) API, interfaz y apertura del puerto de acceso con el laboratorio de redes; (8) automatización y monitoreo.

Resultado esperado: una plataforma que procese SECOP II de forma distribuida, produzca indicadores trazables y permita consultar tanto estadísticas como explicaciones mediante lenguaje natural. La arquitectura es preliminar: el esquema físico definitivo, los pesos del ranking, los umbrales y las decisiones finales de infraestructura deberán validarse durante la implementación.